

X-Teaming Harness

Planner → Attacker → Verifier → TextGrad Optimizer

A faithful, model-agnostic reimplementation of the four-agent multi-turn red-teaming loop from Rahman et al. — for finding multi-turn safety gaps in models you are authorized to test, and generating the data that hardens them.

authorized use only · defensive tooling

Documentation · v0.1.0 · August 2026

Independent implementation of *X-Teaming: Multi-Turn Jailbreaks and Defenses with Adaptive Multi-Agents* (arXiv: 2504.13203). Method by Rahman, Jiang, Shiffer, Liu, Issaka, Parvez, Palangi, Chang, Choi, Gabriel.

CONTENTS

00	Overview
01	Why this design
02	The four core modules
03	The loop, visually
04	Environment setup — start to finish
05	The model matrix (August 2026)
06	Where to get real seed sets
07	Every command, start to finish
08	Evaluating each model out now
09	Analysis commands
10	Reliability & reproducibility
11	Authorized use

00 Overview

This is a defensive tool. It ships with no attack content and no harmful objectives — you supply your own from a sanctioned test set. The whole point of the source paper is its defense dataset, and this harness reproduces that loop end to end.

Single-turn safety is largely solved; multi-turn is not. Harmful intent smeared across a conversation slips past guardrails that would catch it in a single prompt. This harness automates the discovery of those paths: it plans diverse multi-turn attacks, executes them turn by turn against a target, grades every response on a 1–5 scale, and adaptively rewrites a turn when it stalls — then writes reproducible transcripts and optional refusal-substituted training records.

Every role speaks the OpenAI chat protocol, so you point the attacker, target, and verifier at any provider — closed frontier APIs or local open-weight models — and mix them freely.

01 Why this design

Reliably finding multi-turn failures needs three things a single prompt-and-check loop lacks:

- **Diverse planning.** Many genuinely different routes to one objective, not one template reworded. Measured as mean pairwise embedding dissimilarity; the paper reports **0.702** vs. 0.278 for the strongest prior method.
- **A grading signal.** A calibrated 1–5 score on every response, so the system knows whether it is making progress.
- **Adaptive recovery.** When a turn regresses, rewrite it instead of quitting — the **textual-gradient** loop, the paper's key mechanism.

02 The four core modules

CORE 1 · PLANNER Diverse plans Generates N plans per objective across persona / context / strategy / turn-steps. Over-generates, scores pairwise diversity, and regenerates the most redundant until it hits a target (default 0.702). xteaming/planner.py
CORE 2 · ATTACKER Turn-by-turn execution Walks a plan against the target, grades each reply via the Verifier, and hands off to the Optimizer whenever a score drops from the previous turn. xteaming/attacker.py
CORE 3 · VERIFIER 1-5 grading Scores each response: 1 = refusal, 5 = full compliance. Temperature 0 for repeatable grading, a refusal short-circuit for obvious 1s, fails closed. A 5 marks success. xteaming/verifier.py
CORE 4 · OPTIMIZER TextGrad recovery On a score drop, computes a natural-language "textual gradient" (why it underperformed), rewrites the message, re-scores against the live target, keeps the best — up to N_opt = 4. xteaming/optimizer.py

03 The loop, visually

phase 1 Planner 50 diverse plans, ≥ 0.702 pairwise diversity	phase 2 · per turn Attacker composes the next message on-plan, sends to target	score Verifier grades the target reply 1-5	if score drops Optimizer textual gradient → rewrite → re-score, ×4
---	---	---	---

A score of 5 ends the trajectory (objective met). If a plan finishes without a 5, the orchestrator asks the Planner to extend it and retries once. Successful trajectories can be exported with the final harmful turn replaced by a refusal — the XGuard-style defense record.

Verified end-to-end. In testing, a trajectory that scored 3 then dropped to 2 triggered the optimizer and recovered to 5 in a single textual-gradient iteration — the exact adaptive behavior the paper describes.

04 Environment setup — start to finish

BASH · INSTALL

```
# 1. Python 3.10+ in a clean venv
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate

# 2. Install
cd xteaming
pip install -U pip
pip install -r requirements.txt    # openai, numpy, (sentence-transformers)

# 3. Sanity check (offline — no keys)
python -c "import xteaming; print(xteaming.__version__)"
```

A chat subscription is not API access. The harness calls models over the API. Claude Pro/Max (or ChatGPT Plus) is a chat plan with **no API key**, and Anthropic's terms bar driving third-party tools with subscription credentials. To use Claude for any role, get a separate pay-as-you-go **Anthropic API key** from console.anthropic.com. An OpenAI API key alone is enough to run all three roles.

One key, three roles independence rule

Run every role on your one OpenAI key, but use **different models** so the verifier isn't grading its own or the attacker's work. The rule that matters: **Verifier ≠ Attacker, and Verifier ≠ Target.**

BASH · ROLES

```
--attacker gpt-4o-mini  # drives turns; cheap is fine
--target    gpt-4o-mini  # the model under test
--verifier  gpt-4o       # DIFFERENT + stronger grader
```

Fully local with Ollama no keys · offline

```
BASH · LOCAL

ollama serve &                                # start server
ollama pull qwen3.5:27b                        # any tag from $05
export XTEAMING_BASE_URL=http://localhost:11434/v1
export XTEAMING_MODEL=qwen3.5:27b
export OPENAI_API_KEY=ollama                  # any non-empty value
```

05 The model matrix — August 2026

Verify exact ids against each provider's docs before a run — names churn monthly.

PROVIDER	REACH VIA	BASE_URL	EXAMPLE IDS
Anthropic	Anthropic API key	api.anthropic.com/v1/	claude-opus-5 , claude-sonnet-5
OpenAI	native	default	gpt-5.6-sol , gpt-4o , gpt-4o-mini
Google	OpenAI-compatible	.../v1beta/openai	gemini-3-pro , gemini-3-flash
xAI	native	api.x.ai/v1	grok-4
Moonshot	native	api.moonshot.ai/v1	kimi-k3
Open-weight (local)	Ollama	localhost:11434/v1	qwen3.5:27b , llama3.3:70b , deepseek-r1:32b

Every role speaks the OpenAI protocol, so mix providers freely: a strong **attacker** on one endpoint, the **target** on another, a neutral **verifier** on a third to avoid self-grading bias. The CLI's `--verifier-base-url` / `--verifier-key-env` flags let the verifier run fully independent of the attacker.

06 Where to get real seed sets

The shipped objectives are benign. For a real evaluation, supply objectives from a public, peer-reviewed harm benchmark under an authorized engagement. The canonical ones, all public and built for this defensive purpose:

DATASET	WHAT IT IS	WHERE
XGuard-Train	The paper's own 30K multi-turn set, 10K behaviors × 13 categories, with refusal targets	hf: marslabucla/XGuard-Train
HarmBench	400+ standardized harmful behaviors, 7 categories — the de-facto eval set	github: centerforaisafety/HarmBench
AdvBench	Harmful-behavior strings from the GCG paper (Zou et al. 2023)	github: llm-attacks/llm-attacks
JailbreakBench	100 behaviors, harmful + benign paired, with a leaderboard	github: JailbreakBench/jailbreakbench
StrongREJECT	Behaviors + a calibrated grader that avoids over-counting "successes"	github: alexandrasouly/strongreject
MLCommons AILuminate	Standardized safety-grade prompt set across hazard categories	hf: mlcommons/ailuminate

Convert any of them to this harness's format

PYTHON · HARMBENCH → OBJECTIVES.JSONL

```
from datasets import load_dataset      # pip install datasets
import json
ds = load_dataset("walledai/HarmBench", split="train")
with open("objectives.harmbench.jsonl", "w") as fh:
    for row in ds:
        fh.write(json.dumps({"text": row["prompt"],
                             "category": row.get("category", "harmbench")}) + "\n")
```

Some sets require accepting a research-use license on Hugging Face. Use them only within a sanctioned engagement.

07 Every command, start to finish

A complete session from a clean checkout, using the benign objectives. Swap in your own `--objectives` file for a real run.

```
# 0. setup – one OpenAI key covers all three roles
source .venv/bin/activate
export OPENAI_API_KEY=sk-...

# 1. PLAN – generate diverse plans, print realized diversity
python -m xteaming.cli plan \
  --objectives objectives.example.jsonl \
  --model gpt-4o-mini \
  --n-plans 20 --min-diversity 0.702 \
  --out artifacts/plans.json

# 2. ATTACK – full loop, ONE authorized target. Verifier differs from both.
python -m xteaming.cli attack \
  --objectives objectives.example.jsonl \
  --attacker gpt-4o-mini \
  --target gpt-4o-mini \
  --verifier gpt-4o \
  --n-plans 20 --max-plans 5 \
  --out runs --authorized

# 3. INSPECT
ls runs/*/                                # manifest, summary, objective_*/
cat runs/*/summary.json                  # ASR + per-objective stats

# 4. ANALYZE (offline, no model calls)
python -m xteaming.cli analyze asr      --run runs
python -m xteaming.cli analyze scores --run runs
```

First-run cost. Each turn is several API calls (attacker + verifier, plus up to 4 optimizer calls on a score drop). For a cheap first pass use `--n-plans 4 --max-plans 2`, then scale up. Keeping attacker + verifier on `-mini` -class models keeps a run to cents.

08 Evaluating each model out now

The `eval` subcommand runs the same objectives against several targets and writes a comparable tree — a robustness ranking across the current field.


```
export XTEAMING_TARGET_BASE_URL=http://localhost:11434/v1
ollama pull qwen3.5:27b; ollama pull llama3.3:70b; ollama pull deepseek-r1:32b

python -m xteaming.cli eval \
  --objectives objectives.example.jsonl \
  --attacker gpt-4o-mini \
  --targets "qwen3.5:27b, llama3.3:70b, deepseek-r1:32b" \
  --verifier gpt-4o \
  --n-plans 20 --max-plans 5 \
  --out eval/round1 --authorized

python -m xteaming.cli analyze compare --eval eval/round1
```

A defensible protocol: hold attacker, verifier, objective set, `n_plans`, and `max_plans` constant across all targets; vary only the target. Report ASR, mean turns-to-success, and the per-category breakdown. Use a third-party verifier and sanity-check it with `analyze calibration`.

09 Analysis commands

All six run offline over saved runs — no keys, deterministic.

COMMAND	ANSWERS
<code>analyze asr</code>	Attack success rate — overall, per category, mean best score
<code>analyze scores</code>	Per-turn 1-5 curves + where TextGrad engaged and recovered
<code>analyze diversity</code>	Realized plan diversity per objective vs. the 0.702 target
<code>analyze turns</code>	Turns-to-success histogram — attack efficiency
<code>analyze calibration</code>	The verifier's 1-5 distribution — a spike flags a mis-tuned grader
<code>analyze compare</code>	Robustness ranking across an eval/ tree

Five interesting analyses to run

- **Attack-surface heatmap.** Pivot each target's `by_category` ASR into a category × model grid — shows which harm class each model is weakest on.
- **TextGrad lift.** Compare `scores` with the optimizer on vs. off (`optimize_on_drop=False`); the ASR delta is the measured value of the recovery loop.
- **Turn-budget curve.** Re-run `eval` at `--max-plans` 1, 3, 5, 10 and plot ASR vs. budget — the knee is the cheapest setting that still finds most gaps.

- **Verifier-agreement audit.** Grade the same transcripts with two verifier models and diff `calibration` — disagreement flags subjective grading.
- **Diversity-vs-yield.** Correlate each objective's realized diversity with whether it succeeded — tests the paper's claim that more diverse plans find more gaps.

10 Reliability & reproducibility

- **Deterministic where it counts** — verifier at temperature 0 with a fixed seed; every run writes a `manifest.json` capturing models, configs, and seeds.
- **Structured outputs with repair** — strict JSON, one repair pass, then fail closed: a verifier that can't grade returns 1, never a false success.
- **Diversity by construction** — the Planner measures diversity and regenerates the most redundant plans until the target is met or the budget is spent.
- **Full audit trail** — every optimizer iteration (critique, candidate, score) is saved for review and replay.

11 Authorized use

Run this **only** against models you own or have **written permission** to assess, and **only** with objectives from a sanctioned engagement.

- The orchestrator refuses to run until you set `authorized=True` (`--authorized` on the CLI).
- No harmful objectives ship with this code. Source them from the public benchmarks in §06 and tag each to a taxonomy.
- The defense exporter substitutes a refusal for the final harmful turn, so the artifact you keep trains safe behavior rather than distributing elicited content.

Use it to make models safer. That is the entire point of the paper it implements — Rahman et al., *X-Teaming*, COLM 2025 (arXiv:2504.13203).